Тема 6. Теоретическое задание (выполнил Павел Григоренко)

1. Рассмотрим расширение простого типизированного λ-исчисления со ссылками (TaPL 13) операцией

$$\texttt{with}\ x = \texttt{ref}\ t_1\ \texttt{do}\ t_2,$$

при которой создаётся временная ячейка памяти (с начальным значением равным вычисленному значению t_1), доступная только на время выполнения блока t_2 (и освобождающаяся по завершении блока). Покажите, почему даже такая конструкция (без явного оператора `free`) ломает безопасность типизации:

   (a) Выпишите два правила вычисления для конструкции `with–ref` (одно для случая, когда t_1 является значением, другое для случая, когда t_1 не является значением). Правила должны учитывать состояние памяти (μ).

   (b) Предложите правило типизации для конструкции `with–ref`. Правило должно учитывать структуру типизации памяти (Σ).

   (c) Покажите, что сохранение типизации [1, Теорема 13.5.3] не выполняется. Составьте выражение `t`, использующее конструкцию `with–ref`, и представьте состояния памяти μ и μ', контекст Γ, структуры типизации Σ и Σ', а также типы T и T', для которых выполняются условия:

$$\texttt{t} \mid \mu \longrightarrow^* \texttt{t}' \mid \mu',$$

$$\Gamma \mid \Sigma \vdash \mu, \qquad \Gamma \mid \Sigma \vdash \texttt{t} : T, \qquad \Gamma \mid \Sigma' \vdash \mu',$$

   но при этом

$$\Gamma \mid \Sigma' \vdash \texttt{t}' : T', \quad \text{где } T \neq T'.$$

Решение. *место для решения*

2. Реализуйте следующие функции над списками [1, §11.12], используя ссылки [1, §12], и **не** используя `fix` или `letrec` [1, §11.11]:

   (a) `length[T]` : `List[T]` → `Nat`

   (b) `reverse[A]` : `List[A]` → `List[A]`

   (c) `map[A,B]` : (`A` → `B`) → `List[A]` → `List[B]`

   При необходимости используйте стандартные операции со списками из TaPL (например, `nil[T]`, `cons[T] t u`, `isnil[T] t`, `head[T] t`, `tail[T] t`) и доступные операции со ссылками (`ref`, `!`, `:=`, последовательное выполнение).

   Решение.

   (a)

```
length[T] =
let result = ref (λ l : list[T] . 0) in
(
  f := (λ l : list[T] .
    if isnil[T] l
    then 0
    else succ ((!f) (tail[T] l))
  )
  ;
  ! f
)
```

   (b)

```
reverse[T] =
let move = ref (λ to : list[T] . λ from : list[T] . to) in
(
  move := (λ to : list[T] . λ from : list[T] .
    if isnil[T] from
    then to
    else (!move) (cons[T] (head[T] from) to) (tail[T] from)
  )
  ;
  (! move) (nil[T])
)
```

   (c)

```
map[T, U] = λ f : (T -> U) .
let inner = ref (λ l : list[T] . nil[U]) in
(
  inner := (λ l : list[T] .
    if isnil[T] l
    then nil[U]
    else cons[U] (f (head[T] l)) ((!inner) (tail[T] l))
  )
  ;
  ! inner
)
```

Список литературы

[1] Б. Пирс. *Типы в языках программирования: пер. с англ.* Лямбда пресс, 2012. ISBN: 9785791300829. URL: https://books.google.ru/books?id=HJJCkgEACAAJ.